

Магистратура
«Искусственный интеллект»

Инструменты разработки
Лекция 5

Чернов Евгений
НИУ ВШЭ

Сеть: SSH, HTTP, WebSocket, gRPC

Как сервисы общаются по сети: адреса и порты, вход на сервер по SSH и прикладные протоколы — HTTP, WebSocket и gRPC

План занятия

1. **Адрес и порт**
   IP, порт, сокет; кто слушает и на каком адресе
2. **SSH**
   пара ключей, вход без пароля, scp и rsync
3. **Проброс портов**
   -L и -R: туннели через SSH
4. **HTTP**
   запрос и ответ, методы, коды, curl
5. **Real-time**
   polling, long polling, WebSocket
6. **gRPC**
   protobuf, HTTP/2, стриминг, сравнение с REST

01

РАЗДЕЛ 01

Адрес, порт, сокет

Как пакет находит нужную машину и нужную программу на ней

IP-адрес, порт и сокет

IP-адрес находит компьютер в сети, порт — программу на нём (0–65535). Пара «IP + порт» — сокет, конечная точка. TCP-соединение — это пара сокетов: клиента и сервера.

Клиент

`203.0.113.5:51000`

порт выдан системой временно

TCP-соединение: сокет ↔ сокет



запрос → · ← ответ

Сервер

`198.51.100.7:443`

известный порт сервиса

TCP — **соединение, доставка по порядку, повтор потерянного**

HTTP, SSH, WebSocket, gRPC — где важна целостность

UDP — **без соединения и без гарантий, зато быстро**

DNS, видеозвонки, игры — где важна скорость

Кто слушает порт и на каком адресе

Сервер «слушает» порт на конкретном адресе. От адреса зависит, кто до него достучится: только эта машина или вся сеть.

ss -tlnp · столбцы сокращены

```
$ ss -tlnp
State    Local Address:Port    Process
LISTEN   0.0.0.0:8000          python3
LISTEN   127.0.0.1:5432        postgres
LISTEN   [::]:22               sshd
# macOS: lsof -iTCP -sTCP:LISTEN -P -n
```

`0.0.0.0`
все IPv4-интерфейсы — доступен из сети

`127.0.0.1 = localhost`
только с этой же машины

`[::]` · `[::1]`
то же для IPv6: все адреса · localhost

Порты 0–1023 — системные: 22 SSH, 80 HTTP, 443 HTTPS. В Linux их занимает только root, поэтому dev-серверы живут на 8000, 8080, 3000. «Address already in use» — порт уже кем-то занят.

02

РАЗДЕЛ 02

SSH: безопасный доступ к серверу

Пара ключей, вход без пароля, копирование файлов и проброс портов

Асимметричная криптография: пара ключей

SSH-ключи заменяют пароль. Ключей два: приватный хранится только у вас, публичный кладут на сервер. Что подписано приватным ключом, проверяется публичным.

Ваш компьютер

приватный ключ

`~/.ssh/id_ed25519`

секрет: никому не отдавать

`ssh-copy-id`



на сервер уходит только .pub

Сервер

публичный ключ

`~/.ssh/authorized_keys`

не секрет: можно показывать всем

Вход: сервер проверяет, что вы можете подписать данные этой сессии приватным ключом. Сам ключ и пароль по сети не передаются.

Генерация ключа и вход без пароля

терминал (пути сокращены)

```
$ ssh-keygen -t ed25519 -C "you@hse"
Generating public/private ed25519 key pair.
Your identification has been saved in ~/.ssh/id_ed25519
Your public key has been saved in ~/.ssh/id_ed25519.pub

$ ssh-copy-id you@server
Number of key(s) added: 1

$ ssh you@server
# вошли без пароля
```

id_ed25519
приватный, права 600

id_ed25519.pub
публичный, уходит на сервер

authorized_keys
доверенные ключи на сервере

known_hosts
отпечатки серверов у вас

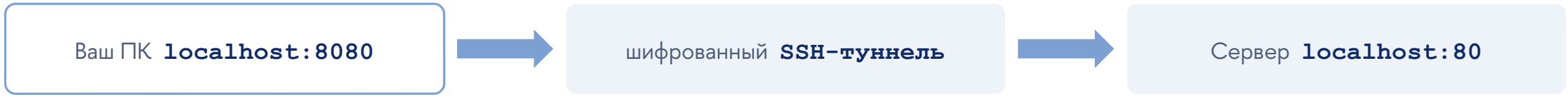
! **Права важны:** папка ~/.ssh — 700, приватный ключ — 600. Если ключ доступен всем, ssh напишет UNPROTECTED PRIVATE KEY FILE и не станет его использовать.

Проброс портов: -L и -R

Проброс пускает TCP-порт через зашифрованный SSH-туннель: удалённый сервис открывается как локальный — или ваш локальный сервис становится виден на сервере.

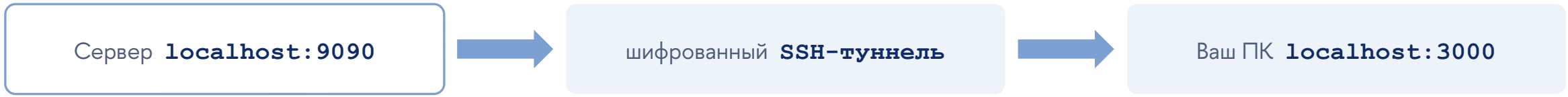
`-L` **Local: порт открывается у вас**

```
ssh -L 8080:localhost:80 you@srv
```



`-R` **Remote: порт открывается на сервере**

```
ssh -R 9090:localhost:3000 you@srv
```



! **Левое число — порт, который откроется;** справа — куда он ведёт. localhost справа — это адрес глазами той стороны, где кончается туннель.

Проброс на практике: база за фаерволом

PostgreSQL на сервере слушает только 127.0.0.1:5432 — снаружи порт закрыт. Подключим к нему свой psql, ничего не открывая в интернет.

```
у себя, два терминала

# 1) туннель; -N — без команды на сервере
$ ssh -N -L 15432:localhost:5432 you@srv

# 2) клиент идёт в свой порт 15432
$ psql -h localhost -p 15432 -U app db
db=> select count(*) from users;
 count
-------
 12480
```

15432
свободный порт у вас: не мешает своей базе

порт 5432 закрыт
наружу торчит только SSH, вход по ключу

так же
Redis, админки, Jupyter на сервере

Кто слушает: при -L порт открывает ssh у вас, при -R — sshd на сервере. По умолчанию оба только на 127.0.0.1: открыть порт -R для всей сети можно лишь с GatewayPorts в настройках sshd.

Копирование файлов: scp и rsync

терминал

```
$ scp report.pdf you@srv:/tmp/
report.pdf    100%  412KB   2.1MB/s   00:00

$ scp -r build/ you@srv:/opt/app/
# -r — папку целиком

$ rsync -avz data/ you@srv:/data/
sending incremental file list
sent 2,145 bytes  received 35 bytes
total size is 26,623  speedup is 12.21
# повторный запуск передаст только изменения
```

scp — **Просто и всегда целиком**

копирует поверх SSH; повторный запуск снова шлёт всё

rsync — **Умно и по разнице**

шлёт только изменённое; -a сохраняет атрибуты, -z сжимает

Хвостовой слэш в rsync

data/ — содержимое папки, **data** — сама папка внутри цели

03

РАЗДЕЛ 03

HTTP

Запрос и ответ, методы и коды, curl как главный инструмент отладки

Анатомия запроса и ответа

В HTTP/1.1 запрос и ответ — обычный текст, устроенный одинаково: стартовая строка, заголовки, пустая строка, тело.

запрос — шлёт клиент

```
GET /users/1 HTTP/1.1
Host: api.example.com
Accept: application/json

# у GET тела нет
```

ответ — шлёт сервер

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 21

{"id":1,"name":"Ann"}
```

1 Стартовая строка

метод · путь · версия или версия · код · текст

2 Заголовки

«Имя: значение» — по одному на строку

3 Пустая строка

сигнал: заголовки закончились

4 Тело

JSON, HTML, файл; у GET обычно нет

Один запрос «на проводе»: curl -v

curl -v показывает весь путь запроса: строки * — служебные, > — что ушло на сервер, < — что пришло обратно.

curl -v · вывод сокращён

```
$ curl -v https://httpbin.org/get
*   Trying 35.170.125.171:443...
* Connected to httpbin.org (35.170.125.171) port 443
* ALPN: server accepted h2
> GET /get HTTP/2
> Host: httpbin.org
> User-Agent: curl/8.4.0
> Accept: */*
< HTTP/2 200
< content-type: application/json
< content-length: 255
* Connection #0 to host httpbin.org left intact
```

*** Trying · Connected**
DNS дал IP, TCP-соединение с :443

*** ALPN · TLS**
шифруем канал, выбрали HTTP/2

> запрос
метод, путь и заголовки от curl

< ответ
код и заголовки; тело — в stdout

left intact
соединение оставлено открытым

Методы и коды ответа

Метод	Путь	Действие
GET	/users	список
GET	/users/42	один пользователь
POST	/users	создать
PUT	/users/42	заменить целиком
PATCH	/users/42	изменить часть полей
DELETE	/users/42	удалить

- **1xx** — **информация** 101 — переход на WebSocket
- **2xx** — **успех** 200 OK · 201 Created · 204
- **3xx** — **перенаправление** 301 · 302 + Location
- **4xx** — **ошибка клиента** 400 · 401 · 403 · 404 · 405
- **5xx** — **ошибка сервера** 500 · 502 · 503

! **REST — соглашение:** путь называет ресурс, метод — действие над ним. По первой цифре кода видно, кто виноват: 4xx — запрос, 5xx — сервер.

curl в работе: код, заголовки, тело

терминал

```
$ curl -s -o /dev/null -w '%{http_code}\n' \
    https://httpbin.org/status/404
404
$ curl -sI https://httpbin.org/get | head -1
HTTP/2 200
$ curl -sIL https://httpbin.org/redirect/1 | grep HTTP
HTTP/2 302
HTTP/2 200
$ curl -s https://httpbin.org/post \
    -H 'Content-Type: application/json' \
    -d '{"name":"Ann"}' | jq -c .json
{"name":"Ann"}
```

-s
тихо, без прогресс-бара

-o /dev/null -w
тело в мусор, печатаем код

-I · -L
только заголовки · идти по Location

-H · -d
заголовок · тело; с -d метод POST

| jq
разобрать и отфильтровать JSON

HTTP/1.1 и HTTP/2

	HTTP/1.1	HTTP/2
Формат	текст	бинарные кадры
Запросы в соединении	по одному за раз	много потоков параллельно
Заголовки	целиком в каждом запросе	сжаты (HPACK), имена строчными
Строка ответа в curl -v	HTTP/1.1 200 OK	HTTP/2 200
Где встретите	везде; http:// без TLS	https-сайты, gRPC

Смысл не изменился: методы, коды и заголовки те же. `curl --http1.1` — принудительно старая версия; без флага curl выбирает сам при TLS-рукопожатии (ALPN).

04

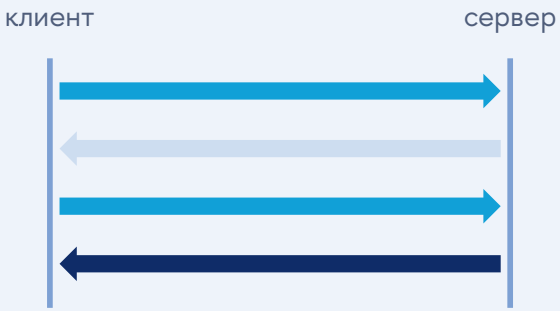
РАЗДЕЛ 04

WebSocket и gRPC

Когда запроса-ответа мало: живой канал в обе стороны и вызовы со строгим контрактом

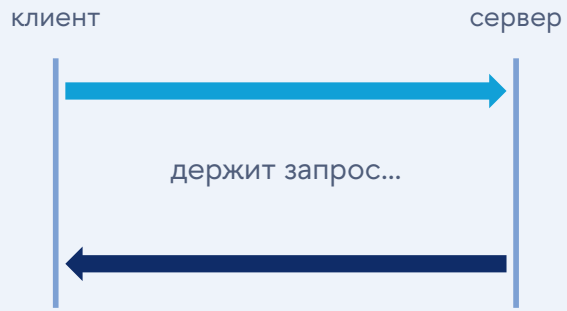
Три модели обновлений в реальном времени

Short polling



клиент спрашивает по таймеру — часто впустую

Long polling



сервер держит запрос, пока не появятся данные

WebSocket



одно соединение: обе стороны шлют, когда хотят

Слева направо — меньше пустых запросов и меньше задержка. Но WebSocket держит соединение открытым: сервер должен помнить каждого клиента.

WebSocket: из HTTP в двусторонний канал

WebSocket начинается как обычный HTTP/1.1-запрос с заголовком Upgrade. Сервер отвечает 101 — и то же TCP-соединение становится каналом, по которому обе стороны шлют сообщения.

рукопожатие · curl -v --http1.1 · сокращено

```
> GET /raw HTTP/1.1
> Host: ws.postman-echo.com
> Connection: Upgrade
> Upgrade: websocket
> Sec-WebSocket-Version: 13
> Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
< HTTP/1.1 101 Switching Protocols
< Upgrade: websocket
< Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

websocat · эхо-сервер

```
$ websocat wss://ws.postman-echo.com/raw
hello       # набрали
hello       # сервер вернул
```

Когда нужен

чаты, уведомления, live-дашборды, игры, совместное редактирование

wss:// — WebSocket поверх TLS, как https://; **ws://** — без шифрования.

Polling по HTTP или WebSocket

	Polling (HTTP)	WebSocket
Кто начинает	всегда клиент	любая сторона, когда есть что сказать
Соединение	запрос → ответ → свободно	живёт минуты и часы
Задержка	до следующего опроса	сразу
Лишний трафик	много пустых ответов	только сообщения с данными
Сервер	stateless, легко масштабировать	держит тысячи открытых соединений
Когда брать	редкие обновления, CRUD	чат, уведомления, живые данные

! **Тот же выбор встретится у Telegram-ботов:** бот получает обновления опросом (polling) или сервер Telegram сам зовёт его (webhook).

gRPC: контракт в .proto

gRPC — удалённый вызов процедур: клиент зовёт метод сервера как обычную функцию. Контракт описан в .proto, по нему генерируется код клиента и сервера.

users.proto

```
syntax = "proto3";

service UserService {
  rpc GetUser (UserId) returns (User);
  rpc ListUsers (ListRequest) returns (stream User);
}

message UserId      { int32 id = 1; }
message ListRequest { int32 limit = 1; }
message User {
  int32  id   = 1;
  string name = 2;
}
```

protobuf
схема данных и компактный бинарный формат

= 1, = 2
номера полей: на проводе вместо имён

stream
поток сообщений вместо одного

HTTP/2
транспорт: много вызовов в одном соединении

Четыре типа вызовов и grpcurl

терминал · публичный grpcb.in

```
$ grpcurl -plaintext grpcb.in:9000 list
addsvc.Add
…
hello.HelloService
$ grpcurl -plaintext -d '{"greeting":"HSE"}' \
    grpcb.in:9000 hello.HelloService/SayHello
{
  "reply": "hello HSE"
}
```

unary · SayHello
один запрос → один ответ

server stream · LotsOfReplies
запрос → поток ответов

client stream · LotsOfGreetings
поток запросов → ответ

bidi · BidiHello
потоки в обе стороны

-plaintext: порт 9000 у grpcb.in без TLS (с TLS — 9001). list и describe работают, только если на сервере включена reflection; иначе grpcurl нужен .proto-файл.

gRPC против REST

	REST	gRPC
Данные	JSON — текст	protobuf — бинарный формат
Контракт	OpenAPI — по желанию	.proto — обязателен
Транспорт	HTTP/1.1 или HTTP/2	HTTP/2
Стриминг	нет встроенного	4 типа вызовов
Из браузера	напрямую	через gRPC-Web и прокси
Отладка	curl, читается глазами	grpcurl, нужна схема

! **На практике — вместе:** gRPC между своими сервисами, REST/JSON наружу — для браузера и партнёров.

ИТОГИ ЛЕКЦИИ

Что важно унести

1 **Адрес и порт**
IP — машина, порт — программа; 0.0.0.0 и 127.0.0.1 — разные адреса

2 **SSH**
вход по ключу без пароля; -L и -R пробрасывают порты

3 **HTTP и curl**
метод, путь, заголовки → код и тело; curl -v видит всё

4 **WebSocket и gRPC**
живой канал в обе стороны · строгий контракт на HTTP/2

Дальше в курсе — **Контейнеризация и виртуализация**: у контейнера своя сеть и свои порты.


ВЫСШАЯ • ШКОЛА • ЭКОНОМИКИ •